Reflexión como Desarrollador Profesional: El Uso de Git en el Proyecto Web Educa

Reydi Cuesta (#05), Adrián Fermín (#13), Ismael Sierra (#34) y Ramón Taveras (#36)

6to E

Diseño e Implementación de Soluciones Web y Multimedia

Prof. Ramón Morillo

19 de octubre de 2026

Reflexión como Desarrollador Profesional: El Uso de Git en el Proyecto Web Educa

El presente trabajo reflexiona sobre la experiencia de incorporar Git, un sistema de control de versiones distribuido, al desarrollo del Proyecto Web Educa. A partir de esa experiencia se analizan los problemas que existían antes de usar la herramienta, la forma en que ayudó a organizar el trabajo, las ventajas de las ramas, el valor del historial de cambios y la relación de todo ello con el trabajo en equipo dentro de la industria del software.

¿Qué Problemas Tenía el Proyecto sin Git?

Antes de usar Git, el proyecto dependía de copias de archivos compartidas por memorias USB, correo o mensajería. Esto provocaba que existieran varias versiones de un mismo archivo con nombres como “index_final”, “index_final2” o “index_corregido”, sin que nadie supiera con certeza cuál era la más reciente. Cuando dos integrantes modificaban el mismo archivo al mismo tiempo, uno sobrescribía el trabajo del otro y se perdían cambios sin que el equipo se diera cuenta.

Además, no había forma de volver a una versión anterior cuando algo dejaba de funcionar, ni de saber quién había hecho un cambio, cuándo ni por qué. Chacon y Straub (2014) señalan que el método de copiar archivos a otras carpetas es común por su sencillez, pero muy propenso a errores, y esa fue exactamente nuestra experiencia: el tiempo que debía dedicarse a programar se perdía en reunir, comparar y corregir archivos.

¿Cómo Ayudó Git a Organizar el Trabajo?

Git nos dio un repositorio central en GitHub, donde se encuentra la versión oficial del proyecto. Cada integrante clona ese repositorio, trabaja en su computadora y luego sube sus aportes, de manera que todos parten siempre de la misma base. Los cambios se guardan en *commits*, que son puntos de control con un mensaje que explica lo que se hizo, lo cual obliga a dividir el trabajo en pasos pequeños y comprensibles.

Gracias a esto el trabajo quedó repartido por responsabilidades: una parte del equipo se ocupó de la interfaz, otra del módulo de registro y otra de la sección de estudiantes. Cuando Git detecta que dos personas modificaron las mismas líneas, señala el conflicto en lugar de borrar

uno de los cambios, y el equipo decide cómo combinarlos. Así se eliminó la confusión sobre cuál era la versión correcta.

¿Qué Ventajas Aporta el Uso de Ramas?

Una rama es una línea de desarrollo independiente que permite trabajar en una funcionalidad sin afectar la versión estable del proyecto (Chacon y Straub, 2014). En el Proyecto Web Educa se crearon las ramas *interfaz*, *registro* y *estudiantes*, además de la rama principal *main*. Esto permitió que cada integrante avanzara en su parte de forma paralela, sin esperar a los demás ni romper el código de sus compañeros.

Las ramas también reducen el riesgo de experimentar: si una idea no funciona, basta con descartar la rama y *main* permanece intacta. Cuando una funcionalidad está terminada y probada, se fusiona (*merge*) con la rama principal, idealmente mediante una solicitud de integración (*pull request*) que otro integrante puede revisar antes de aceptarla (GitHub, s.f.). De este modo la rama principal contiene siempre una versión funcional que se puede presentar o publicar.

¿Por Qué Es Importante el Historial de Cambios?

El historial de Git registra cada *commit* con su autor, su fecha y su descripción. Esto funciona como la memoria del proyecto: permite saber qué cambió, quién lo hizo y con qué propósito. Si aparece un error, se puede revisar el historial para encontrar el momento en que se introdujo y, si es necesario, regresar a una versión anterior que sí funcionaba, sin rehacer el trabajo desde cero.

El historial también aporta responsabilidad y transparencia, porque cada aporte queda asociado a la persona que lo realizó; esto facilita evaluar la participación de cada integrante y entender decisiones tomadas semanas atrás. Para que sea útil, los mensajes de *commit* deben ser claros y descriptivos, ya que un historial con mensajes como “cambios” o “arreglos” pierde gran parte de su valor (Loeliger y McCullough, 2012).

¿Cómo Se Relaciona Esto con el Trabajo en Equipo Real?

En las empresas de desarrollo de software, Git es una herramienta estándar y el flujo que practicamos es el mismo que se usa profesionalmente: un repositorio compartido, ramas por

funcionalidad, revisiones de código mediante *pull requests* e integración hacia una rama principal estable. En esos entornos trabajan decenas o cientos de personas sobre el mismo código, muchas veces desde lugares y horarios distintos, por lo que sin control de versiones la coordinación sería imposible.

Esta experiencia nos preparó para ese contexto. Aprendimos a comunicarnos por medio de mensajes de *commit*, a resolver conflictos en lugar de evitarlos, a no modificar directamente la versión estable y a revisar el trabajo de los compañeros. Más allá de los comandos, comprendimos que el control de versiones es una práctica de colaboración y disciplina profesional, y que dominarlo es una competencia fundamental para cualquier desarrollador que aspire a trabajar en equipos reales.

Referencias

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

GitHub. (s.f.). *Acerca de las ramas*. GitHub Docs. https://docs.github.com/es/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches

Loeliger, J., y McCullough, M. (2012). *Version control with Git* (2.ª ed.). O'Reilly Media.